

Vertex Asset Manager Specification

Version: 1.0

Status: Draft

1. Objetivo

O Asset Manager é o subsistema do Vertex responsável por gerenciar todos os recursos estáticos utilizados pelo sistema e pelos plugins.

Seu objetivo é fornecer carregamento eficiente, cache inteligente, versionamento e isolamento de assets como:

- imagens
- ícones
- fontes
- arquivos de UI
- layouts declarativos
- recursos de tema
- bundles de plugin

2. Responsabilidades

O Asset Manager é responsável por:

- carregar assets sob demanda;
- gerenciar cache de recursos;
- versionar assets por plugin e Core;
- otimizar uso de memória para assets;
- fornecer acesso seguro e isolado;
- integrar assets com UI Runtime;
- compactar e descompactar bundles;
- invalidar cache quando necessário;
- resolver conflitos de versão;
- monitorar uso de recursos visuais.

3. Arquitetura

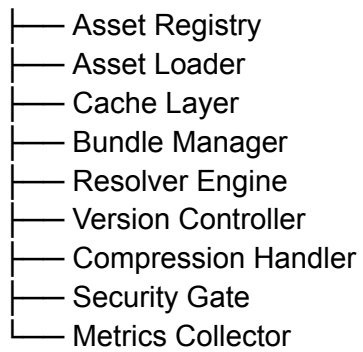
Plugin / Core

↓

Asset API



Asset Manager



Storage Engine

4. Componentes

Asset Registry

Mantém o catálogo de todos os assets disponíveis no sistema.

Asset Loader

Carrega assets sob demanda.

Cache Layer

Armazena assets frequentemente usados para acesso rápido.

Bundle Manager

Gerencia pacotes de assets de plugins e Core.

Resolver Engine

Resolve conflitos entre assets com mesmo nome.

Version Controller

Controla versões de assets e compatibilidade.

Compression Handler

Comprime e descomprime assets para otimizar armazenamento.

Security Gate

Controla acesso a assets sensíveis.

Metrics Collector

Monitora uso e performance de assets.

5. Fluxos

Carregamento de asset

Plugin / UI Runtime

↓

Asset API

↓

Security Gate

↓

Resolver Engine



Asset Loader



Cache / Storage



Return Asset

Cache hit

Request



Cache Layer



Return Immediate

Bundle de plugin

Plugin Install



Bundle Manager



Asset Registry



Storage Engine

Invalidação de asset

Version Change



Version Controller



Cache Invalidation



Reload Assets

6. APIs

Carregamento

- loadAsset()
- loadImage()
- loadFont()
- loadBundle()

Cache

- cacheAsset()
- invalidateAsset()
- clearCache()

Consulta

- getAsset()
- listAssets()
- assetExists()

Bundle

- registerBundle()
- unregisterBundle()

7. Estados

Estado do asset

Not Loaded

↓

Loading

↓

Cached

↓

Active

↓

Invalidated

↓

Removed

Estado de bundle

- Installed
- Active
- Updating
- Deprecated
- Removed

8. Políticas

Lazy loading

Assets só devem ser carregados quando necessários.

Cache inteligente

Assets frequentemente usados devem ser mantidos em cache.

Isolamento por plugin

Assets de plugins não podem interferir entre si.

Controle de memória

Cache deve respeitar limites do Memory Management.

Versionamento obrigatório

Assets devem ser versionados para evitar conflitos.

Segurança de acesso

Assets sensíveis requerem validação do Security Framework.

9. Integrações

O Asset Manager integra-se com:

- UI Runtime Engine;
- UI Engine;
- Plugin Manager;
- Plugin Context;
- Storage Engine;
- Memory Management;
- Resource Manager;
- Security Framework;
- Configuration System;
- Logging Framework;
- Execution Runtime;
- Boot Manager.

10. Métricas

O sistema coleta:

- taxa de cache hit/miss;
- tempo de carregamento de assets;
- uso de memória por asset;
- assets carregados por plugin;
- número de invalidações;
- conflitos de versão;
- tempo de resolução de assets;
- uso de bundles.

11. Exemplos

Carregamento de ícone de UI

1. UI Runtime solicita asset.
2. Asset Manager verifica cache.
3. Asset não encontrado → Loader ativa.
4. Asset é carregado e armazenado.
5. UI recebe recurso.

Cache hit

1. Plugin solicita imagem.
2. Cache Layer encontra asset.
3. Retorno imediato.
4. Sem acesso ao Storage Engine.

Atualização de tema

1. Novo bundle de assets é instalado.
2. Version Controller invalida cache antigo.
3. Assets são recarregados.
4. UI Runtime aplica novos recursos.

Conflito de assets

1. Dois plugins usam mesmo nome de asset.
2. Resolver Engine aplica regra de prioridade.
3. Asset correto é selecionado por contexto.

12. Regras Arquiteturais

- Assets devem ser carregados sob demanda.
- Cache deve ser controlado pelo Memory Management.
- Plugins não podem acessar assets de outros plugins diretamente.
- Versionamento é obrigatório para todos os assets.
- UI Runtime não deve acessar Storage diretamente.
- Segurança deve validar assets sensíveis.

13. Limitações

O Asset Manager não é responsável por:

- execução de código;
- lógica de UI;
- armazenamento de dados estruturados;
- agendamento de tarefas;
- gerenciamento de plugins.

14. Futuras Extensões

O Asset Manager poderá evoluir para:

- streaming de assets em tempo real;
- compressão adaptativa baseada em bateria;
- pré-carregamento inteligente por contexto;
- sincronização entre dispositivos Vertex;
- assets dinâmicos gerados em runtime;
- otimização por GPU.

15. Resumo

O Asset Manager é o sistema central de gerenciamento de recursos visuais e estáticos do Vertex. Ele fornece carregamento eficiente, cache inteligente, versionamento e isolamento de assets para plugins e Core, garantindo desempenho, segurança e consistência visual em toda a plataforma.